List of Abbreviations

Abbreviation	Full Name
5G	Fifth Generation
OFDM	Orthogonal Frequency Division Multiplexing
MIMO	Multiple-Input Multiple-Output
FFT	Fast Fourier Transform
ZF	Zero-Forcing
MMSE	Minimum Mean Square Error
CSI	Channel State Information
MF	Matched Filter
HWPE	Hardware Processing Engine

List of Symbols

Symbol	Full Name
$\mathbf{Y}$	Received signal vector
$\mathbf{H}$	Channel matrix
$\mathbf{H^H}$	Hermitian transpose of the channel matrix
$\mathbf{X}$	Transmitted signal vector
$\mathbf{X\hat{}}$	Estimated transmitted signal vector
$\mathbf{N}$	Additive noise vector
$\mathbf{A}$	Gram matrix
$\mathbf{b}$	Matched-filter output vector
$\mathbf{L}$	Lower triangular matrix obtained from Cholesky decomposition
$\mathbf{L^H}$	Hermitian transpose of the lower triangular matrix
$\mathbf{z}$	Intermediate solution vector
$\mathbf{N_t}$	Number of transmit antennas
$\mathbf{N_r}$	Number of receive antennas
$\mathbf{L_{solve}}$	Latency of the triangular-system solving process
$\mathbf{A^{-1}}$	Inverse of the Gram matrix
$\mathbf{l_{ij}}$	Element of the lower triangular matrix
$\mathbf{x_i}$	Element of the transmitted signal vector
$\mathbf{z_i}$	Element of the intermediate solution vector
$\mathbf{B_i}$	Element of the matched-filter output vector

Chapter 10

1 Hardware I: Algorithms

1.1 Introduction

As discussed in **"MIMO in Mobile Communication Systems"**, multiple-input multiple-output (MIMO) systems employ multiple transmit and receive antennas to improve spectral efficiency and communication reliability. Although this architecture provides significant performance advantages, the transmitted signals become coupled through the wireless channel, making signal recovery at the receiver a challenging task. Consequently, an equalization stage is required to separate the transmitted data streams and mitigate the effects of channel distortion. [1][ch10]

Within the considered MIMO-receiver, equalization is performed after FFT processing and before symbol detection. The FFT converts the received signals from the time domain into the frequency domain, while the equalizer uses channel information to estimate the transmitted symbols associated with each subcarrier. The accuracy of this stage directly influences the reliability of the subsequent detection process.[1][ch10]

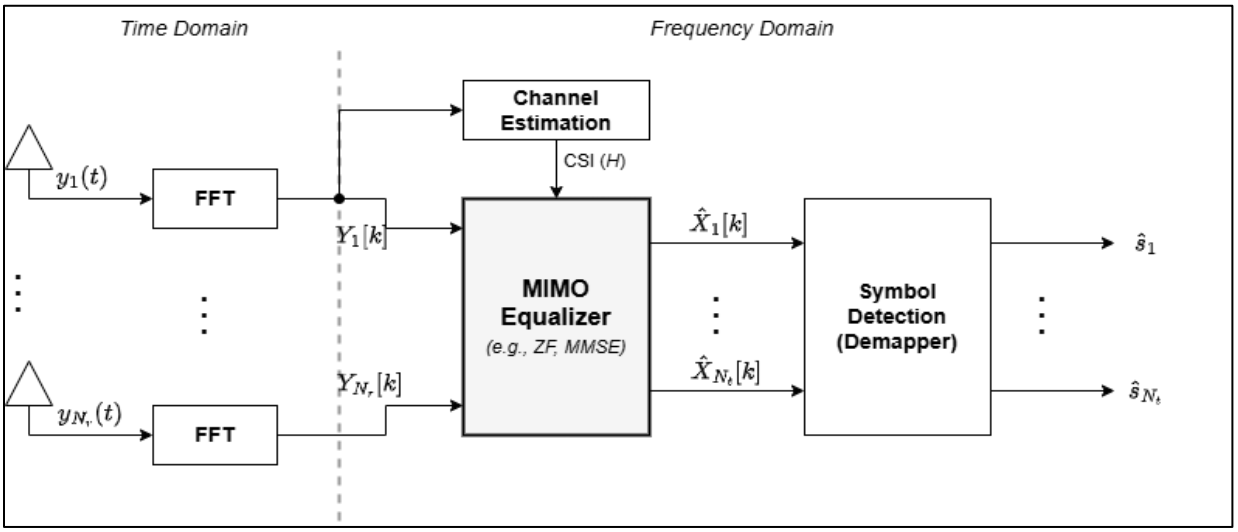

Figure 1.1: Position of the equalizer within the MIMO-OFDM receiver chain (redrawn based on [Ref.1] [ch10].).

The equalization problem can be formulated as a system of linear equations relating the transmitted symbols, the wireless channel, and the received observations. Several techniques have been proposed to solve this problem, including Zero-Forcing (ZF) and Minimum Mean Square Error (MMSE) equalization. These approaches typically require solving matrix equations whose complexity increases with the number of antennas and transmitted streams. [1], [2][ch10]

For practical MIMO systems, direct matrix inversion can become computationally expensive and difficult to implement efficiently in hardware. As a result, alternative formulations are often adopted to reduce computational complexity while maintaining equivalent mathematical functionality. One widely used approach consists of transforming the equalization problem into a sequence of triangular-system solutions through

Cholesky decomposition. This reformulation eliminates the need for explicit matrix inversion and provides a structure that is well suited for hardware implementation. [2] [3][ch10]

The work presented in this chapter focuses on the algorithmic foundation of the equalizer adopted in this thesis. The equalization problem is first formulated in matrix form and subsequently transformed into a Cholesky-based solution. The resulting algorithm is then decomposed into forward and backward substitution stages, which form the basis of the hardware architecture developed later in **"Hardware II: HWPE"** and integrated into the complete receiver pipeline presented in **"Hardware III: System Pipeline"**.

1.2 MIMO Equalization Problem

In a MIMO communication system, multiple data streams are transmitted simultaneously through the wireless channel. During propagation, each receive antenna observes a superposition of the signals transmitted from all antennas. Consequently, the received signals contain both the desired information and the interference introduced by the coupling between spatial streams. Recovering the transmitted symbols therefore requires a processing stage capable of separating these mixed signal components. [1][ch10]

The relationship between the transmitted and received signals can be represented using this system model

$$Y = HX + N \quad \textbf{(1.1)}$$

where:

- Y is the received signal vector,
- H is the channel matrix,
- X is the transmitted signal vector,
- N is the additive noise vector.

The channel matrix (H) characterizes the propagation conditions between every transmit and receive antenna pair. Each element of the matrix represents the gain and phase shift experienced by a transmitted signal as it travels through the wireless channel. [1][ch10]

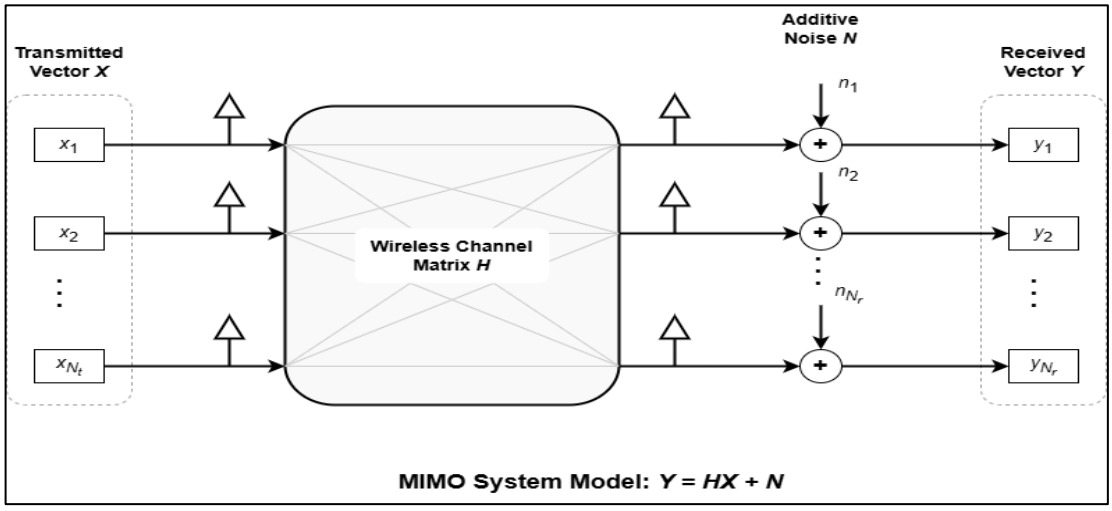

Figure 1.2: MIMO system model (redrawn based on [Ref. 1] [ch10]).

The objective of equalization is to estimate the transmitted signal vector (X) using the available observations (Y) and the known channel matrix (H). This process effectively compensates for channel distortion and mitigates the interference generated by the simultaneous transmission of multiple spatial streams. The resulting estimate is then forwarded to the symbol detection stage for further processing. [1] [ch10]

Several equalization techniques have been proposed for solving this problem. Among the most common approaches are Zero-Forcing (ZF) and Minimum Mean Square Error (MMSE) equalization. Both methods attempt to recover the transmitted symbols by solving a matrix equation derived from the channel model. While their performance characteristics differ, they share a common computational challenge: the need to solve a system of linear equations involving the channel matrix. [1], [2] [ch10]

For an N x N MIMO system, the computational complexity of this operation increases rapidly with the number of antennas. Directly computing the inverse of the channel matrix is often undesirable due to its computational cost and implementation complexity. These limitations become particularly important in hardware-oriented designs, where arithmetic resources, latency, and memory requirements must be carefully controlled. [2] [ch10]

Rather than performing explicit matrix inversion, the equalizer adopted in this work reformulates the detection problem into a linear-system solving problem. This reformulation enables the use of matrix factorization techniques that provide a more structured computational flow and a significantly more hardware-friendly implementation. The mathematical development leading to this formulation is presented in the following section.

Method	Main Objective	Matrix Operation
Zero-Forcing (ZF)	Eliminate inter-stream interference	Matrix inversion
MMSE	Balance interference suppression and noise enhancement	Matrix inversion
Proposed Approach	Solve equivalent linear system	Cholesky decomposition and triangular solving

Table 1.1: Comparison of linear equalization approaches. (Compiled based on [Ref.1] [ch10] and [Ref. 2].)

1.3 From Matrix Inversion to Linear System Solving

The equalization problem introduced in the previous section can be solved by determining the transmitted signal vector (X) from the received observations (Y) and the channel matrix (H). A direct solution may be obtained by manipulating the system model presented in Equation (1.1).[2] [ch10]

To obtain a more structured formulation, the received signal model is first multiplied by the Hermitian transpose of the channel matrix. This operation produces the normal equations

$$H^H H X = H^H Y \quad \textbf{(1.2)}$$

where:

- H^H is the Hermitian transpose of the channel matrix,

The multiplication by H^H transforms the original detection problem into an equivalent linear system while preserving the desired solution. This formulation is widely used in MIMO equalization because it enables the application of efficient matrix factorization techniques. [2][ch10]

For simplicity, the Gram matrix and matched-filter output are defined as

$$A = H^H H \quad \textbf{(1.3)}$$

where: A is the Gram matrix,

Similarly,

$$b = H^H Y \quad \textbf{(1.4)}$$

where: b is the matched-filter output vector,

Using these definitions, the equalization problem can be rewritten as

$$AX = b \quad \textbf{(1.5)}$$

This representation transforms the equalization problem into a standard system of linear equations. The objective is no longer to directly invert the channel matrix, but rather to determine the vector (X) that satisfies the linear system. [2] [ch10]

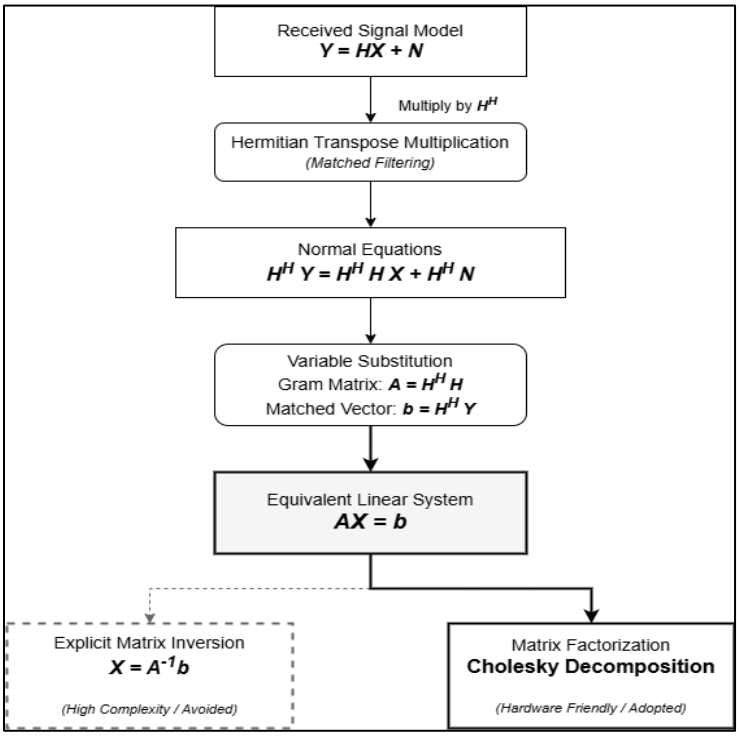

Figure 1.3: Reformulation of the MIMO equalization problem into a linear system (adapted from concepts presented in [Ref. 2] and [Ref. 3] [ch10])

A direct solution of Equation (1.5) may still be obtained through

$$X = A^{-1}b \quad \textbf{(1.6)}$$

where:

- X is the transmitted signal vector,
- A^{-1} is the inverse of the Gram matrix,
- b is the matched-filter output vector.

Although mathematically straightforward, explicit matrix inversion is generally avoided in practical implementations due to its computational complexity and hardware cost. Matrix inversion requires a large number of arithmetic operations and introduces irregular data dependencies that complicate hardware realization. These limitations become increasingly significant as the number of antennas increases.[2] [3] [ch10]

Instead of explicitly computing A^{-1}, the approach adopted in this work solves the linear system through matrix factorization. By decomposing the Gram matrix into a product of triangular matrices, the equalization problem can be transformed into a sequence of simpler operations that are more suitable for hardware implementation. Among the available factorization methods, Cholesky decomposition provides an efficient solution for the Hermitian positive-definite matrices encountered in MIMO equalization. The fundamentals of this decomposition are presented in the following section.

Approach	**Main Operation**	**Hardware Suitability**
Direct Solution	Matrix inversion (A^{-1})	Low
Linear-System Solution	Matrix factorization and substitution	High

Table 1.2: Comparison between direct matrix inversion and linear-system solving approaches (compiled based on [Ref. 2] and [Ref. 3] [ch10]).

1.4 Matched Filtering

The transformation of the equalization problem into the linear system

$$H^H H X = H^H Y$$

introduces two quantities that play a central role in the proposed equalizer: the Gram matrix $H^H H$ and the matched-filter output vector $H^H Y$. Before the system can be solved through Cholesky decomposition and triangular substitution, both quantities must first be computed. [2] [ch10]

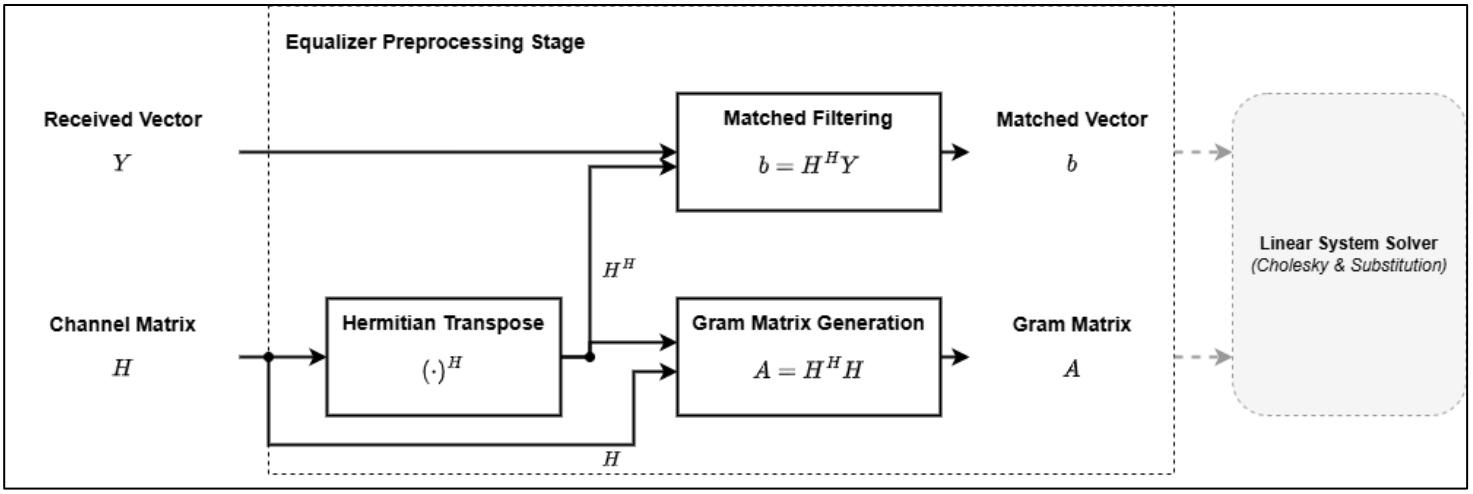

Figure 1.4: Matched-filter operation (redrawn based on [Ref. 2] [ch10]).

Matched filtering generates the vector

$$b = H^H Y$$

where:

- b is the matched-filter output vector,
- H^H is the Hermitian transpose of the channel matrix,
- Y is the received signal vector.

This operation projects the received signal onto the channel basis and combines the information received across all antennas into a form suitable for symbol estimation. As a result, the matched-filter output serves as the right-hand side of the linear system that will later be solved by the equalizer. [2] [ch10]

For an N x N MIMO system, the matched-filter operation can be expressed as

$$b = \begin{bmatrix} h_1^H \\ h_2^H \\ \vdots \\ h_N^H \end{bmatrix} Y \qquad \textbf{(1.7)}$$

where:

- h_i^H is the Hermitian transpose of the channel vector associated with the i^{th} spatial stream,
- Y is the received signal vector,
- b is the matched-filter output vector.

Each element of the resulting vector is obtained through a complex inner product between a channel vector and the received signal vector. Consequently, the matched-filter stage consists primarily of complex multiplications followed by accumulation operations. [2] [ch10]

In addition to generating the vector (b), the equalizer computes the Gram matrix

$$A = H^H H$$

where:

- A is the Gram matrix,
- H^H is the Hermitian transpose of the channel matrix,
- H is the channel matrix.

The Gram matrix captures the correlation between the spatial channels and forms the coefficient matrix of the linear system. Together, the matched-filter output vector and the Gram matrix provide all information required for the subsequent equalization stages. [2] [ch10]

The outputs of the matched-filter stage are subsequently used to construct the linear system

$$AX = b$$

where the Gram matrix (A) forms the coefficient matrix and the matched-filter output (b) forms the right-hand-side vector. The solution of this system is obtained through Cholesky decomposition, which is presented in the following section.

1.5 Cholesky Decomposition

Following the generation of the Gram matrix (A) and the matched-filter output vector (b), the equalization problem is represented by the linear system

$$AX = b \quad \textbf{(1.8)}$$

where:

- A is the Gram matrix,
- X is the transmitted signal vector,
- b is the matched-filter output vector.

The efficient solution of this system is a central requirement in practical MIMO receivers.Although a solution can be obtained through explicit matrix inversion, this approach remains computationally expensive and is generally unsuitable for hardware-oriented implementations. [2] [ch10]

A more efficient alternative is to factorize the matrix (A) into simpler components and solve the resulting systems sequentially. Since the Gram matrix (A) is Hermitian and positive definite under normal operating

conditions, it can be decomposed using Cholesky factorization. This decomposition transforms the original system into a form that can be solved using triangular-system operations. [3] [ch10]

The Cholesky decomposition of the matrix (A) is expressed as

$$A = L\,L^H \tag{1.9}$$

where: L is a lower triangular matrix,

The resulting lower triangular matrix contains all the information required to reconstruct the original matrix while significantly simplifying the solution process. Unlike general matrix factorization methods, Cholesky decomposition exploits the properties of Hermitian matrices to reduce computational complexity and storage requirements. [3][ch10]

Substituting the decomposition into the linear system yields

$$L\,L^H\,X = b \tag{1.10}$$

To simplify the solution process, an intermediate vector z is introduced such that

$$L\,z = b \tag{1.11}$$

where: z is an intermediate solution vector,

Once z has been determined, the transmitted signal vector can be recovered from

$$L^H\,X = z \tag{1.12}$$

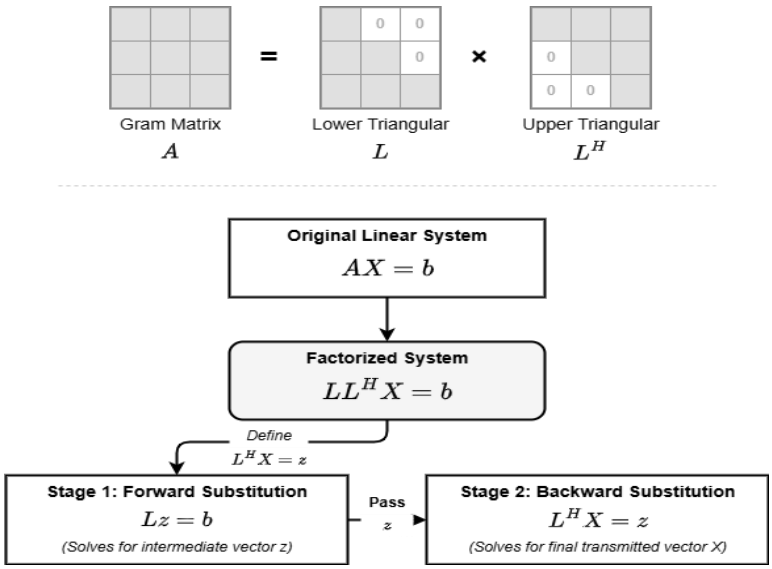

Figure 1.5: Cholesky decomposition of a Hermitian matrix (adapted from [Ref. 3] [ch10]).

This reformulation transforms a single complex matrix problem into two triangular-system solving problems. The first system is solved through forward substitution, while the second is solved through backward substitution. Both operations exhibit a regular computational structure that is particularly attractive for hardware implementation. [3] [ch10]

Cholesky decomposition therefore converts the equalization problem into a sequence of structured computational stages that avoid explicit matrix inversion while preserving the original solution. The resulting triangular systems form the basis of the solving procedure adopted in this work. The forward- and backward-substitution operations used to solve these systems are presented in the following sections.

1.5.1 Forward Substitution

Following Cholesky decomposition, the original equalization problem is transformed into two triangular systems. The first of these systems is represented by

$$L\,z\;=\;b$$

where:

- L is the lower triangular matrix obtained from Cholesky decomposition,
- z is the intermediate solution vector,
- b is the matched-filter output vector.

Since L is a lower triangular matrix, all elements located above the main diagonal are equal to zero. This property introduces a structured dependency pattern that enables the unknown variables to be computed sequentially from top to bottom. [3] [ch10]

For an N x N system, the matrix equation can be written as

$$\begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{N1} & l_{N2} & \cdots & l_{NN} \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_N \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix} \qquad \textbf{(1.13)}$$

where:

- l_{ij} represents an element of the lower triangular matrix,
- z_i is an element of the intermediate solution vector,
- b_i is an element of the matched-filter output vector.

The first unknown can be computed directly because it depends only on the first equation:

$$z_1 = \frac{b_1}{l_{11}} \qquad \textbf{(1.14)}$$

where:

- z_1 is the first intermediate solution,
- b_1 is the first matched-filter output element,
- l_{11} is the first diagonal element of the lower triangular matrix.

Once z_1 has been computed, it can be used to determine z_2. The same procedure continues until all elements of the vector z have been obtained. The general forward-substitution equation is

$$z_i = \frac{1}{l_{ii}}\left(b_i - \sum_{j=1}^{i-1} l_{ij}\, z_j \right) \qquad \textbf{(1.15)}$$

The computation of each element depends on previously calculated values. Consequently, the solution process follows a strict sequential order. A variable cannot be computed until all variables on which it depends have already been determined. [3] [ch10]

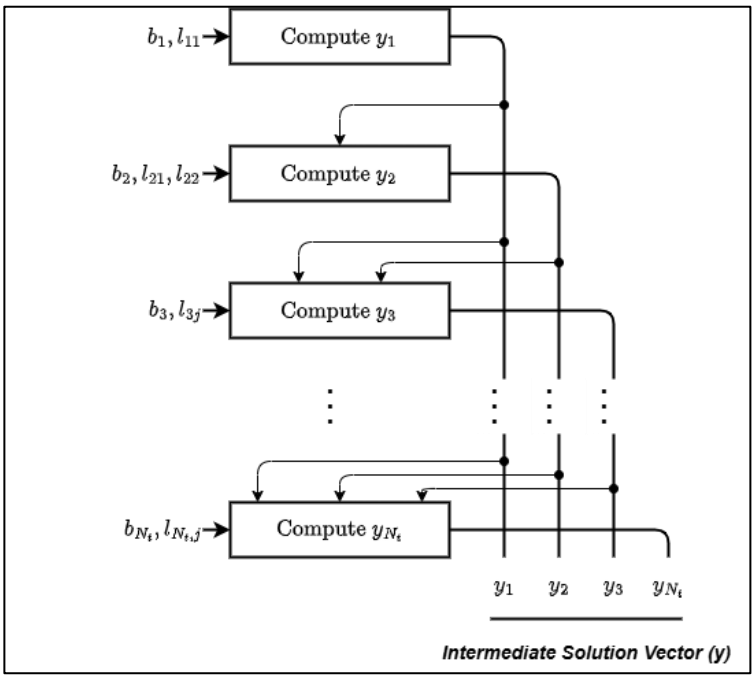

Figure 1.6: Dependency graph of the forward-substitution process.

The regular dependency structure simplifies hardware scheduling and memory access organization. The computational sequence is deterministic, memory accesses follow a predictable order, and the required arithmetic operations consist primarily of multiply-accumulate and division operations. These characteristics simplify scheduling and facilitate efficient hardware implementation. [2] [ch10]

In the equalizer adopted in this work, forward substitution constitutes the first solving stage following Cholesky decomposition. The generated intermediate vector (z) is subsequently used by the backward-substitution stage to recover the estimated transmitted symbols. The operation of this second stage is presented in the next section.

1.5.2 Backward Substitution

After completing the forward-substitution stage, the intermediate solution vector z becomes available. The final estimate of the transmitted signal vector is then obtained by solving the second triangular system

$$L^H X = z$$

where:

- L^H is the Hermitian transpose of the lower triangular matrix,
- X is the transmitted signal vector,
- z is the intermediate solution vector obtained from forward substitution.

Since L^H is an upper triangular matrix, all elements below the main diagonal are equal to zero. This structure allows the unknown variables to be computed sequentially, but in the reverse order compared to forward substitution. [3] [ch10]

For an (N x N) system, the matrix equation can be expressed as

$$\begin{bmatrix} l_{11}^* & l_{21}^* & \cdots & l_{N1}^* \\ 0 & l_{22}^* & \cdots & l_{N2}^* \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & l_{NN}^* \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_N \end{bmatrix} \quad \textbf{(1.16)}$$

where:

- l_{ij}^* denotes the complex conjugate of the corresponding element in L,
- x_i is an element of the transmitted signal vector,
- z_i is an element of the intermediate solution vector.

The solution process begins with the last equation because it contains only a single unknown. The final element of the transmitted signal vector is therefore computed as

$$x_N = \frac{z_N}{l_{NN}^*} \quad \textbf{(1.17)}$$

Once x_N has been determined, the remaining variables can be computed successively by moving upward through the system. The general backward-substitution equation is

$$x_i = \frac{1}{l_{ii}^*}\left(z_i - \sum_{j=i+1}^{N} l_{ji}^* \, x_j \right) \quad \textbf{(1.18)}$$

where:

- x_i is the current transmitted signal estimate,
- l_{ii}^* is the diagonal element of the upper triangular matrix,
- l_{ji}^* represents the upper triangular coefficients,
- z_i is the corresponding intermediate solution element.

Similar to forward substitution, each computation depends on values obtained during previous iterations. However, the dependency direction is reversed. The solution process therefore proceeds from the last variable toward the first variable. [3] [ch10]

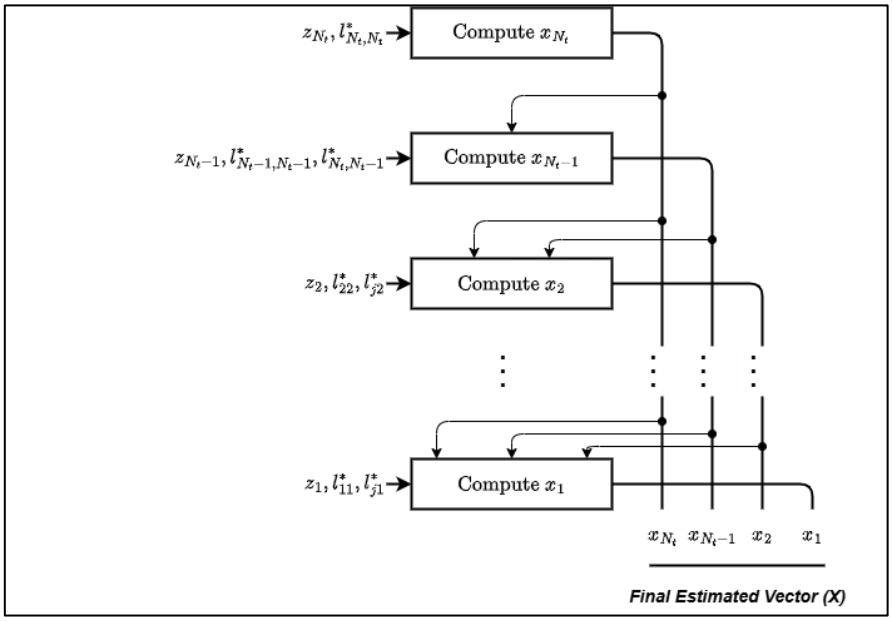

Figure 1.7: Dependency graph of the backward-substitution process.

Backward substitution represents the final solving stage and produces the estimated transmitted symbol vector. Together, the forward- and backward-substitution stages replace the need for explicit matrix inversion while preserving the mathematical solution of the original equalization problem. [2] [ch10]

The sequence formed by matched filtering, Gram matrix generation, Cholesky decomposition, forward substitution, and backward substitution constitutes the complete equalization algorithm adopted in this work. The integration of these stages into a unified processing flow is presented in the following section.

1.6 Proposed Equalizer Algorithm

The previous sections introduced the individual computational stages required to solve the MIMO equalization problem. In practice, these stages operate as a unified processing chain that transforms the received frequency-domain samples into estimates of the transmitted symbols.

The equalizer adopted in this work assumes that the wireless channel remains constant during the processing of an OFDM frame and that the channel matrix HHH is available beforehand. Consequently, channel estimation is outside the scope of this work, and the equalization process begins directly from the received signal vector and the known channel matrix.

The processing flow consists of matched filtering, Gram matrix generation, Cholesky decomposition, forward substitution, and backward substitution. Together, these stages solve the linear system derived in the previous sections without requiring explicit matrix inversion.

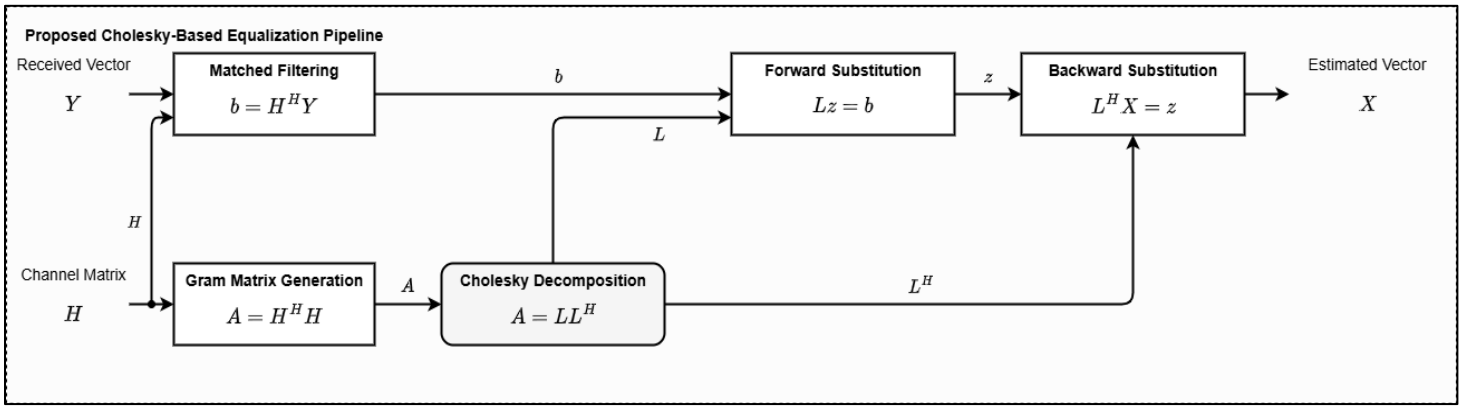

Figure 1.8: Processing flow of the proposed equalization algorithm

The complete sequence can be summarized as

$$Y \rightarrow H^H Y \rightarrow H^H H \rightarrow LL^H \rightarrow L z = b \rightarrow L^H X = z \qquad \textbf{(1.19)}$$

where:

- Y is the received signal vector,
- $H^H Y$ is the matched-filter output vector,
- $H^H H$ is the Gram matrix,
- LL^H is the Cholesky factorization of the Gram matrix,
- z is the intermediate solution vector,
- X is the estimated transmitted signal vector.

The matched-filter stage generates the vector $b = H^H Y$, while the Gram matrix generation stage computes $A = H^H H$. Together, these operations provide the information required to construct the linear system solved by the equalizer. The resulting Gram matrix is then factorized using Cholesky decomposition, producing the lower triangular matrix required for the subsequent solving stages.

Following factorization, forward substitution computes the intermediate solution vector z, which is then used by the backward-substitution stage to recover the transmitted symbol estimates. Since each stage depends on the outputs of the preceding stage, the algorithm naturally forms a structured processing pipeline. The functionality of the individual stages is summarized in the following table.

Stage	Inputs	Outputs	Function
Matched Filtering	(H, Y)	(b = $H^H Y$)	Generate matched-filter output vector
Gram Matrix Generation	(H)	(A = $H^H H$)	Generate Gram matrix
Cholesky Decomposition	(A)	(L)	Matrix factorization
Forward Substitution	(L, b)	(z)	Compute intermediate solution
Backward Substitution	(L^H, z)	(X)	Recover transmitted symbols

Table 1.3: Processing stages of the proposed equalization algorithm.

By avoiding explicit matrix inversion, the adopted algorithm transforms the equalization problem into a sequence of matrix factorization and triangular-solving operations that are more suitable for hardware implementation. Consequently, the algorithm serves as the foundation for the hardware architectures developed later in **"Hardware II: HWPE"** and integrated into the complete receiver presented in **"Hardware III: System Pipeline"**.

1.7 Latency Analysis

In addition to computational complexity, execution latency represents an important characteristic of the proposed equalization algorithm. Although the Cholesky-based formulation eliminates the need for explicit matrix inversion, the solution process remains constrained by the data dependencies inherent in the triangular-system solving stages. Understanding these dependencies is therefore essential for evaluating the algorithm and motivating the hardware architectures presented later in "Hardware II: HWPE".

As discussed in **"Forward Substitution"**, the intermediate solution vector (z) is generated sequentially. Each element depends on previously computed values, preventing the complete solution vector from being calculated simultaneously. Consequently, the computation progresses through the vector in a step-by-step manner, with each iteration producing a new solution element.

A similar behavior is observed in **"Backward Substitution"**, where the transmitted signal vector is recovered from the intermediate solution vector. In this case, the computation proceeds in the reverse direction, beginning with the last element and progressing toward the first. Since each solution element depends on values generated in previous iterations, the execution remains sequential despite the relatively simple arithmetic operations involved.

For an N x N system, the dependency chain associated with the forward-substitution stage requires approximately (N-1) propagation steps. The backward-substitution stage exhibits an equivalent dependency structure and therefore introduces a similar latency contribution. As a result, the overall latency of the triangular-system solving process scales linearly with the system dimension.

By combining the latency contributions of both stages, the total solving latency can be approximated as

$$L_{solve} = 2N - 1 \quad \textbf{(1.20)}$$

where:

- L_{solve} is the latency of the triangular-system solving process,
- N is the dimension of the MIMO system.

Depending on the scheduling methodology and the treatment of initialization and termination operations, the total latency may alternatively be expressed as

$$L_{solve} = 2N - 2 \quad \textbf{(1.21)}$$

channel matrix H is available beforehand, Channel estimation is therefore outside the scope of this work. Consequently, channel estimation is not considered within the scope of this workBoth expressions originate from the dependency patterns described in **"Forward Substitution"** and **"Backward Substitution"** and differ only in the accounting of boundary operations. The key observation is that the latency grows linearly with the system dimension, resulting in a predictable execution behavior that is particularly attractive for hardware implementation.

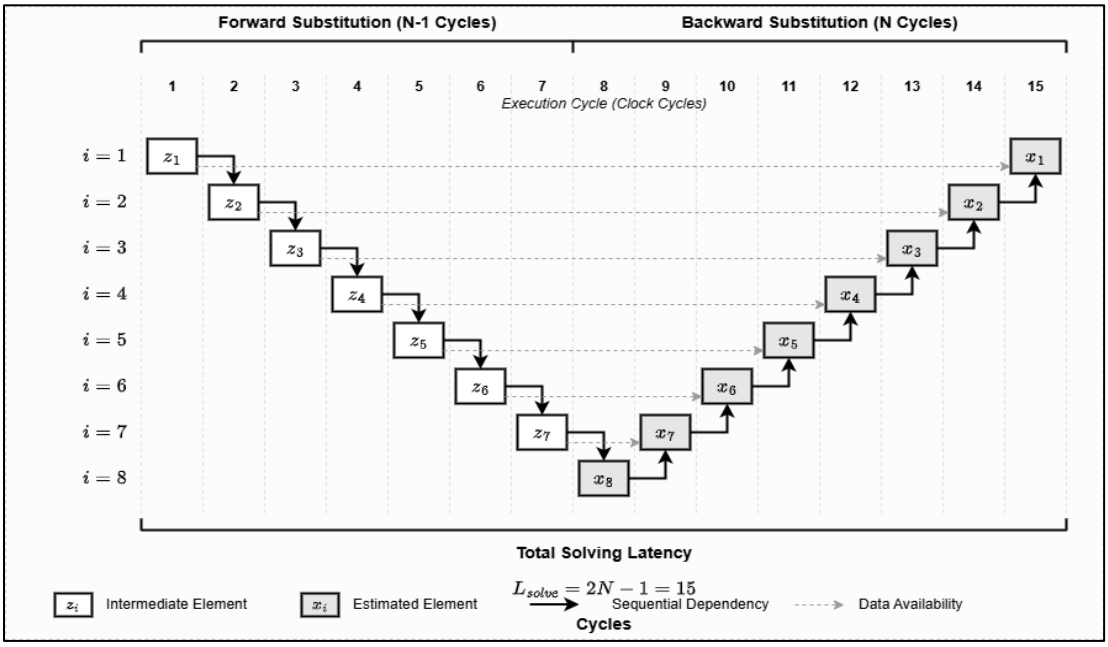

Figure 1.9: Combined execution schedule of the triangular-system solving stages.

The latency model highlights the impact of data dependencies on the execution time of the equalizer. More importantly, the regular execution pattern revealed by this analysis forms the foundation for the hardware scheduling strategies adopted in **"Hardware II: HWPE"**, where dedicated processing elements are developed to efficiently execute the equalization algorithm.

2 References

[1] D. Tse and P. Viswanath, *Fundamentals of wireless communication*. Cambridge: Cambridge University Press, 2005. doi: 10.1017/CBO9780511807213.

[2] F.-L. Luo and C. Zhang, Eds., *Signal processing for 5G: algorithms and implementations*. Chichester, West Sussex: IEEE Press, Wiley, 2016.

[3] G. H. Golub and C. F. Van Loan, *Matrix computations*, 4. ed. in Johns Hopkins studies in the mathematical sciences. Baltimore, MD: Johns Hopkins Univ. Press, 2013.